

GESTÃO DE QUALIDADE DE SOFTWARE



GESTÃO DE CONFIGURAÇÃO E CONTROLE DE VERSÃO: CONCEITOS E PRÁTICAS PARA A MANUTENÇÃO DE SOFTWARE



Introdução



- ✓ A gestão de configuração e o controle de versão são **pilares fundamentais** para a **manutenção e evolução de sistemas** de software.
- ✓ Essas práticas garantem a **integridade** do código-fonte, **facilitam** a **colaboração** entre equipes e ajudam a **rastrear mudanças** ao longo do tempo.

Com o crescente número de colaboradores e a complexidade dos sistemas modernos, ferramentas e práticas de controle de versão se tornaram indispensáveis para garantir a **qualidade**, **rastreabilidade** e **segurança** do código.

O que é Gestão de Configuração de Software (GCS)?



A GCS é um processo sistemático para gerenciar mudanças em um sistema de software, garantindo que o sistema permaneça consistente, funcional e rastreável ao longo do tempo.



Principais atividades da GCS:



Para pensar...



Você está trabalhando em um projeto com mais 3 desenvolvedores ao mesmo tempo e precisa repartir as novas funcionalidades do sistema com eles.

Como você faz a junção do código ao final do desenvolvimento? (Lembrando que o mesmo arquivo pode ser editado por mais de um dev)

Seu chefe pediu para você deletar uma funcionalidade do sistema que não é utilizada. Após 3 meses ele decidiu que quer essa funcionalidade de volta.

Para pensar...



Como você faz para recuperar essa funcionalidade e implementá-la no projeto mesmo após muitas outras mudanças?

Você é muito otimista e por isso não possui nenhum sistema de backup automático da sua aplicação em ambiente de desenvolvimento. Um belo dia seu computador queima e você não havia copiado suas últimas features para um pendrive.

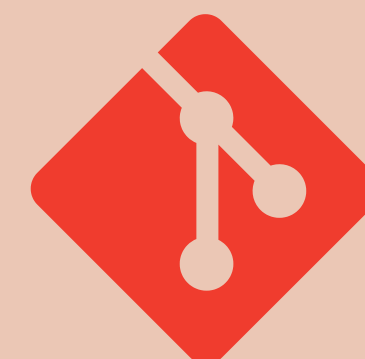
Como você faz para recuperar esse código?

Sistema de Controle de Versão

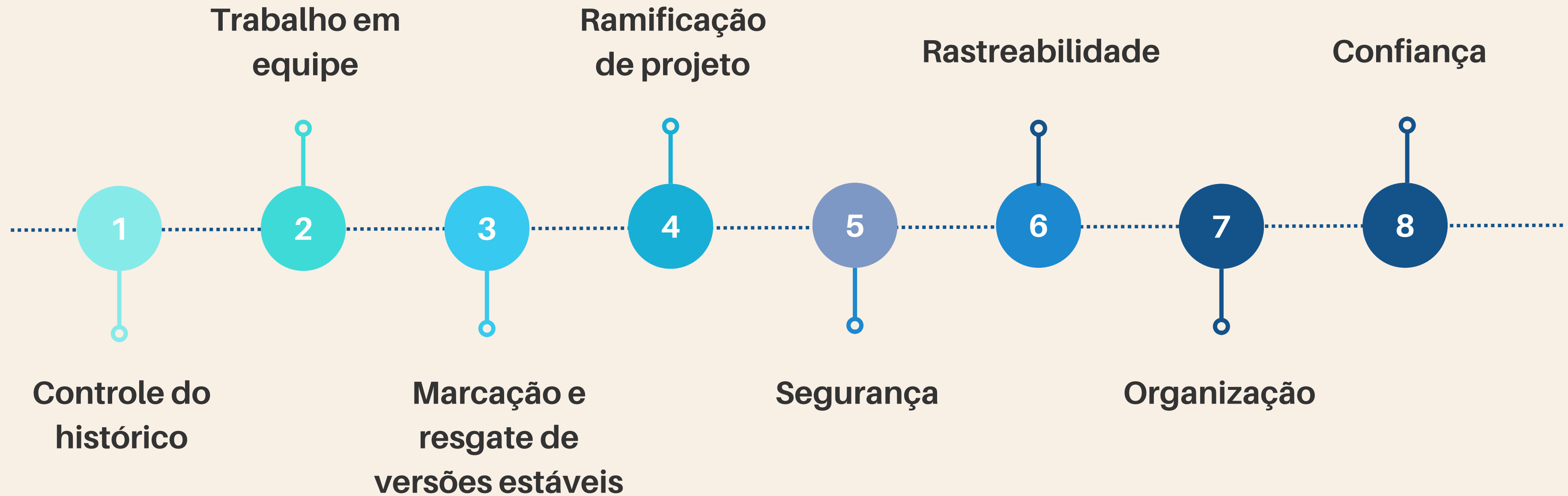


Controle de versão é a prática de **gerenciar diferentes versões** de um software ao longo do tempo. Ele permite que equipes trabalhem **simultaneamente** no código **sem conflitos** e possibilita a **recuperação de versões** anteriores se necessário.

Esses sistemas são comumente utilizados no desenvolvimento de software para controlar as diferentes versões – histórico e desenvolvimento – dos códigos-fontes e também da documentação.



Controle de Versão - Benefícios

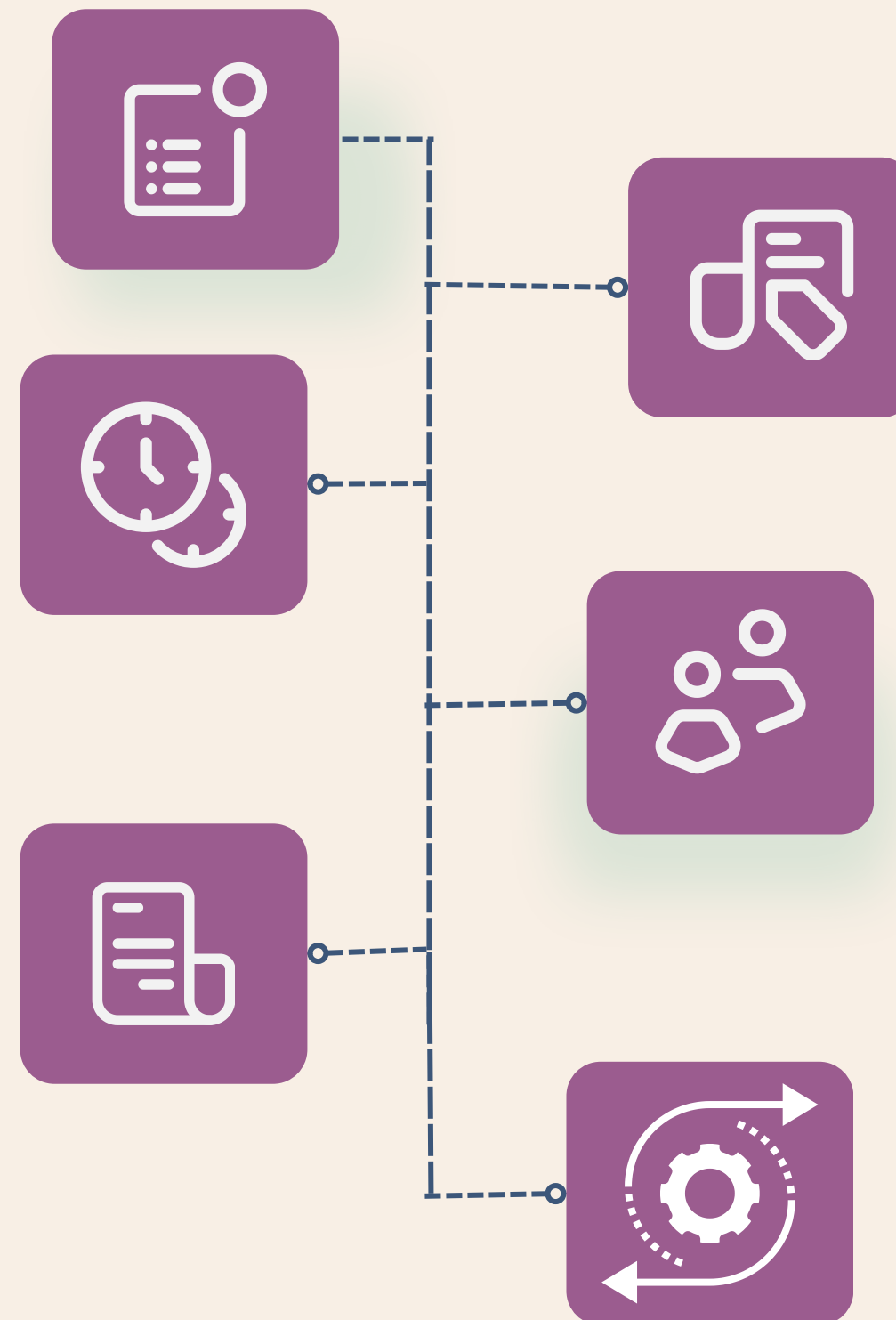


Controle de Versão - Funcionamento Básico

As informações - com todo o histórico - ficam guardadas em um repositório, em um servidor.

O repositório armazena um conjunto de documentos - de modo persistente (sistema de arquivos ou banco de dados).

Cada desenvolvedor possui em sua máquina uma cópia local somente da última versão de cada documento.



Um cliente pode acessar o repositório pela rede ou localmente.

Um repositório possui um endereço lógico que permite a conexão do cliente.

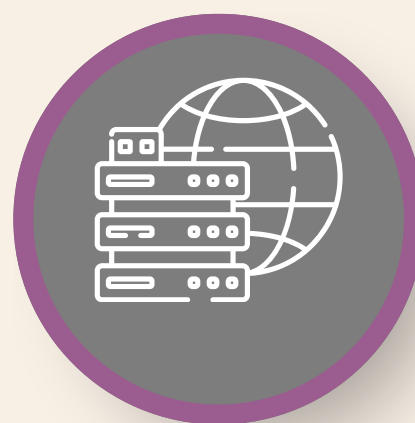
A cada alteração relevante do desenvolvedor é necessário "atualizar" as informações do servidor submetendo as alterações.

Tipos de Controle de Versão



LOCAL

Mantido no próprio dispositivo.



CENTRALIZADO

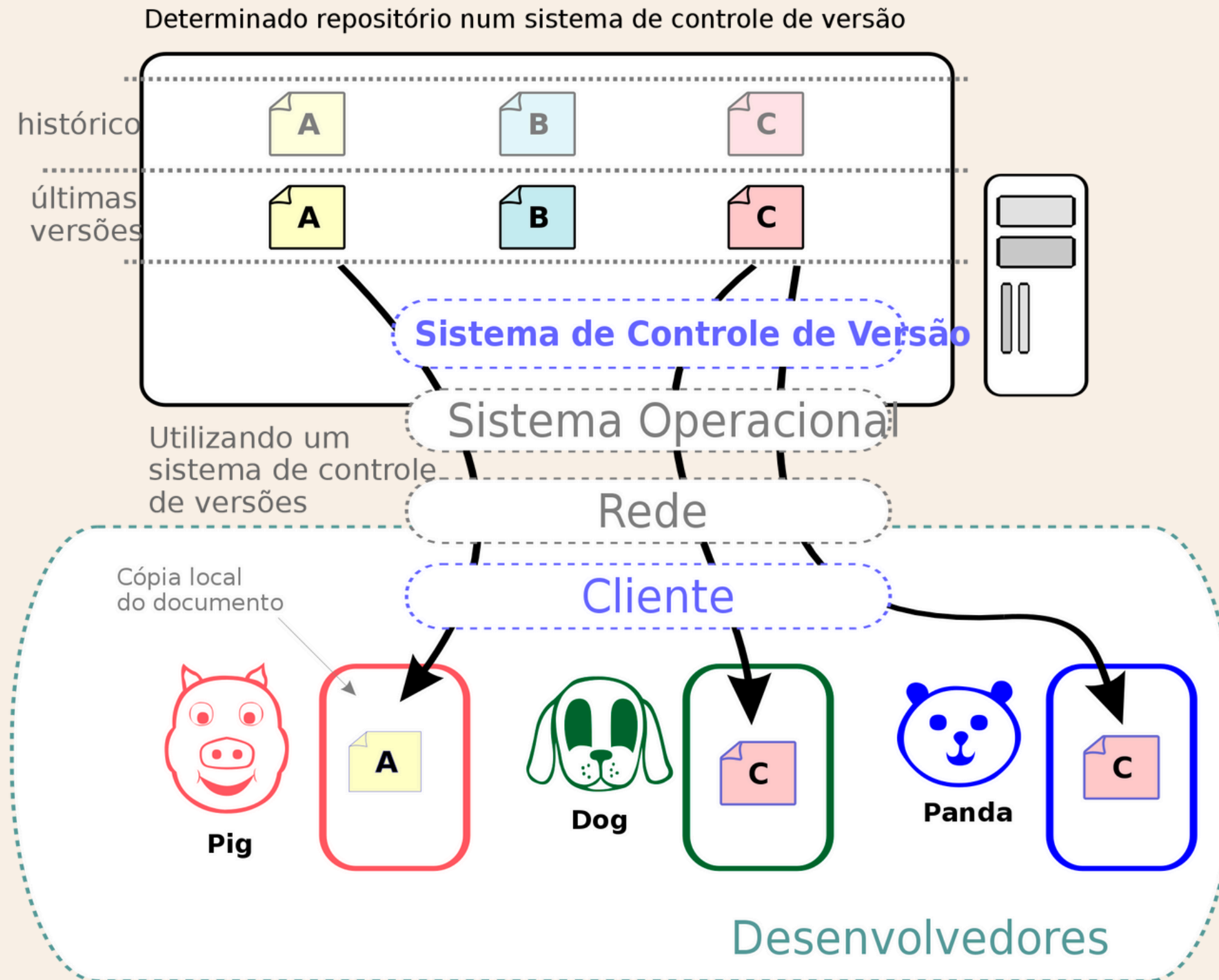
Um servidor central armazena todas as versões.



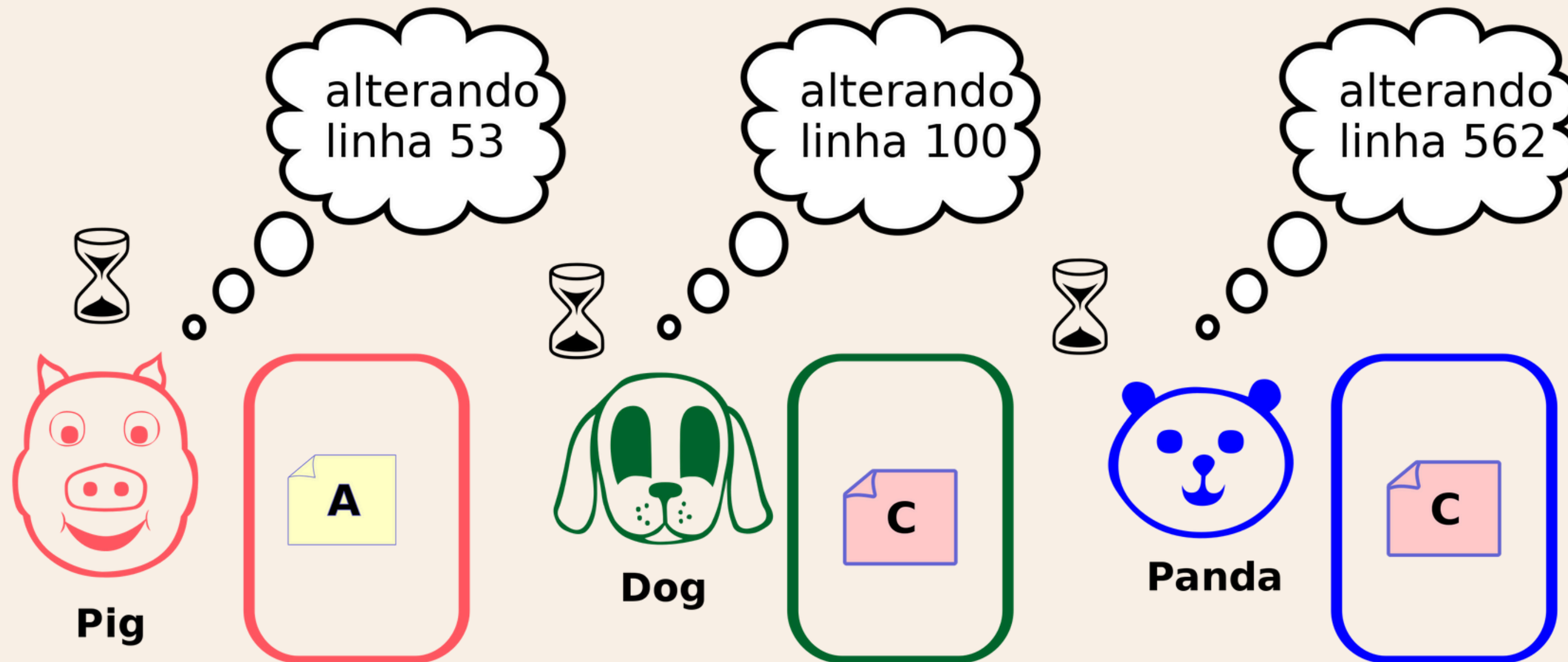
DISTRIBUÍDO

Cada usuário tem uma cópia completa do repositório.

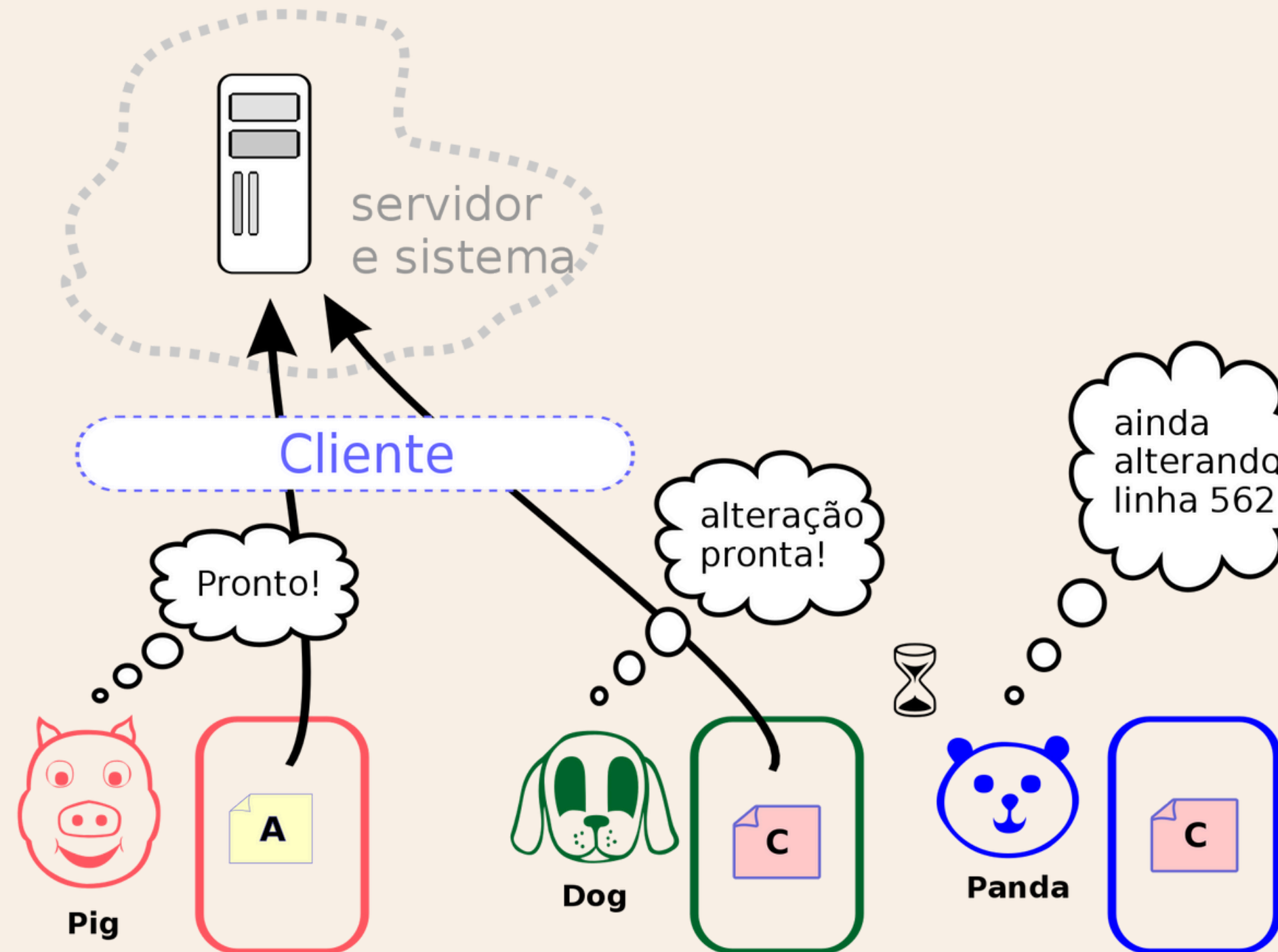
Controle de Versão - Branch and Merge



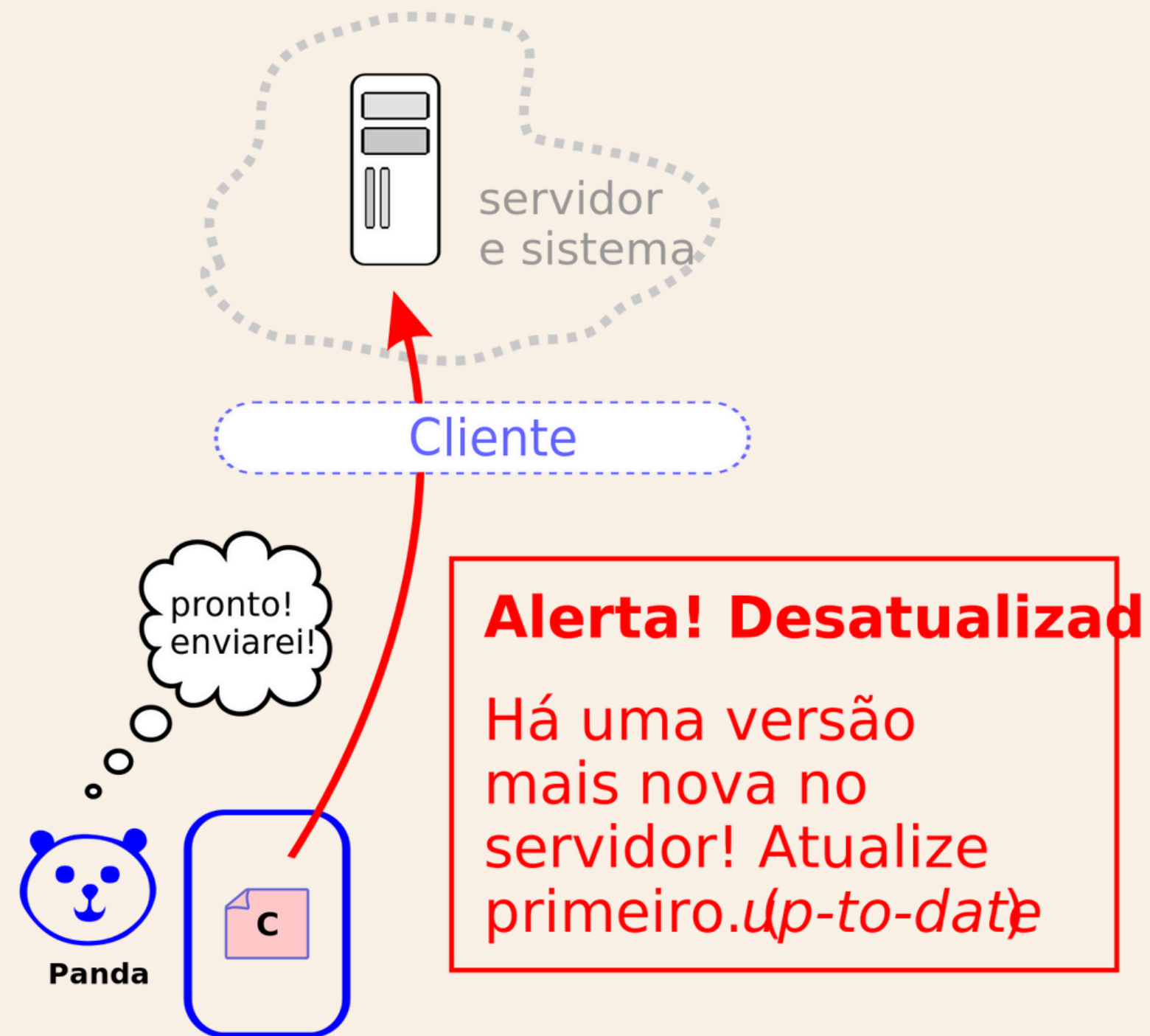
Controle de Versão - Branch and Merge



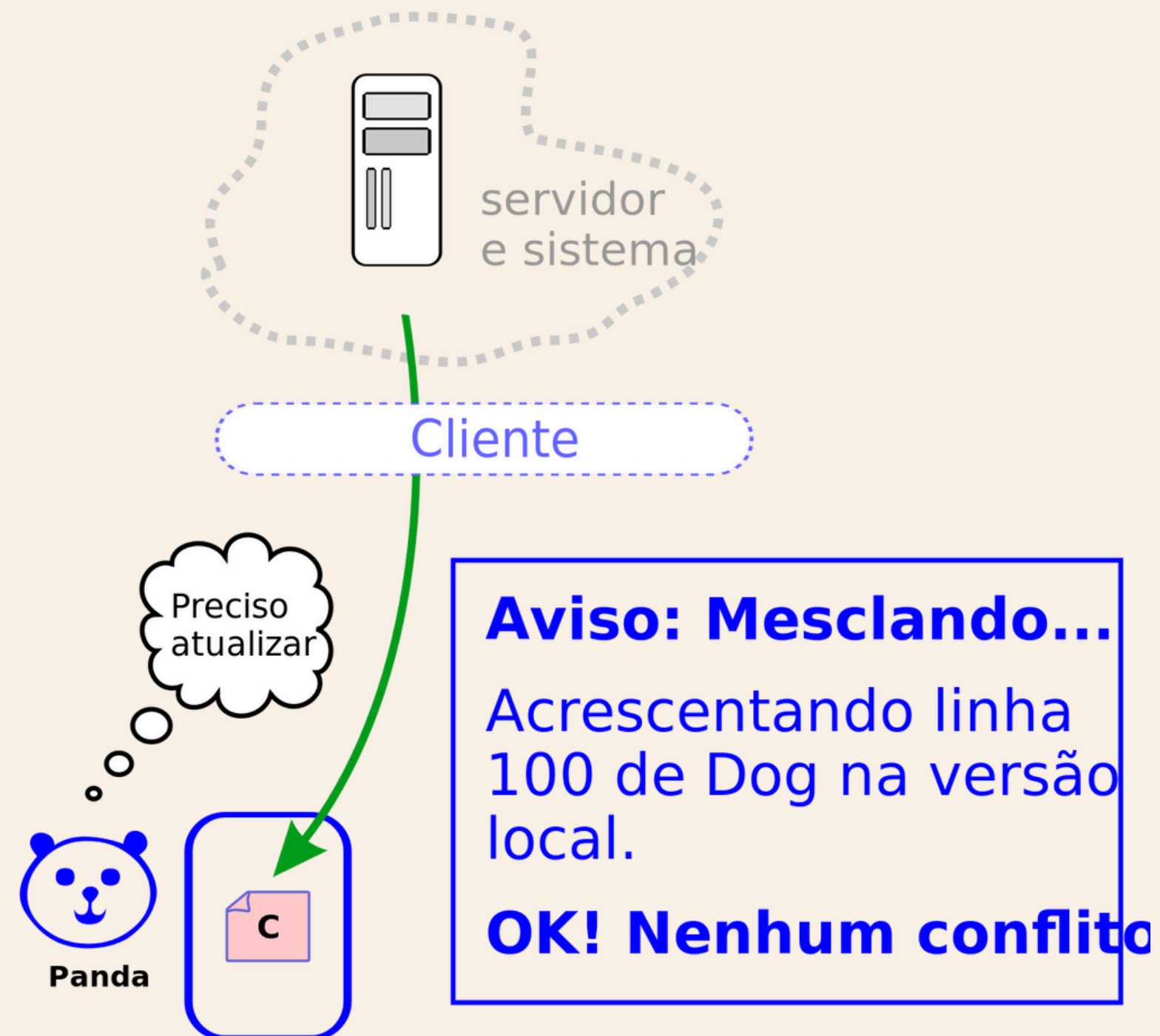
Controle de Versão - Branch and Merge



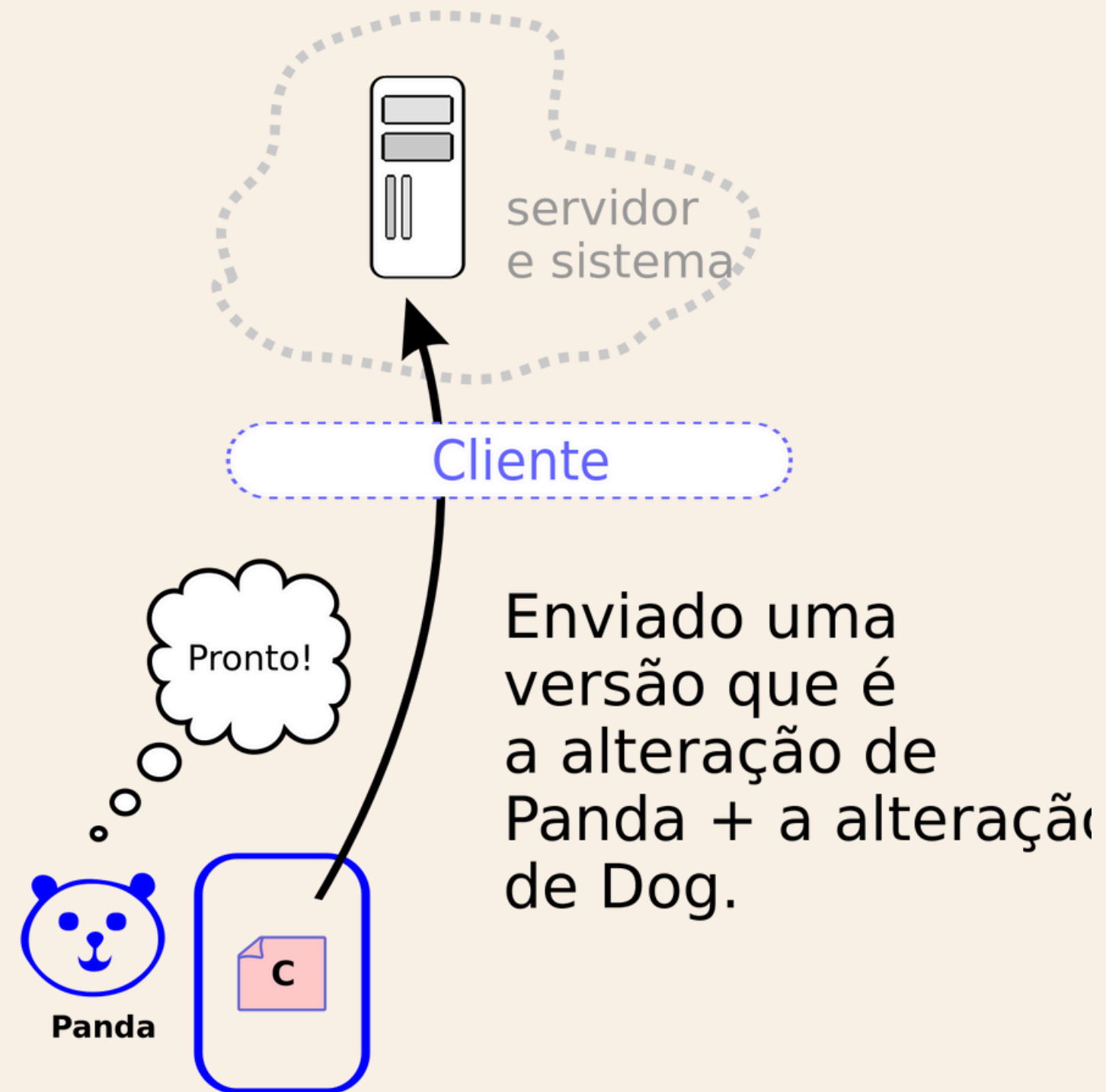
Controle de Versão - Branch and Merge



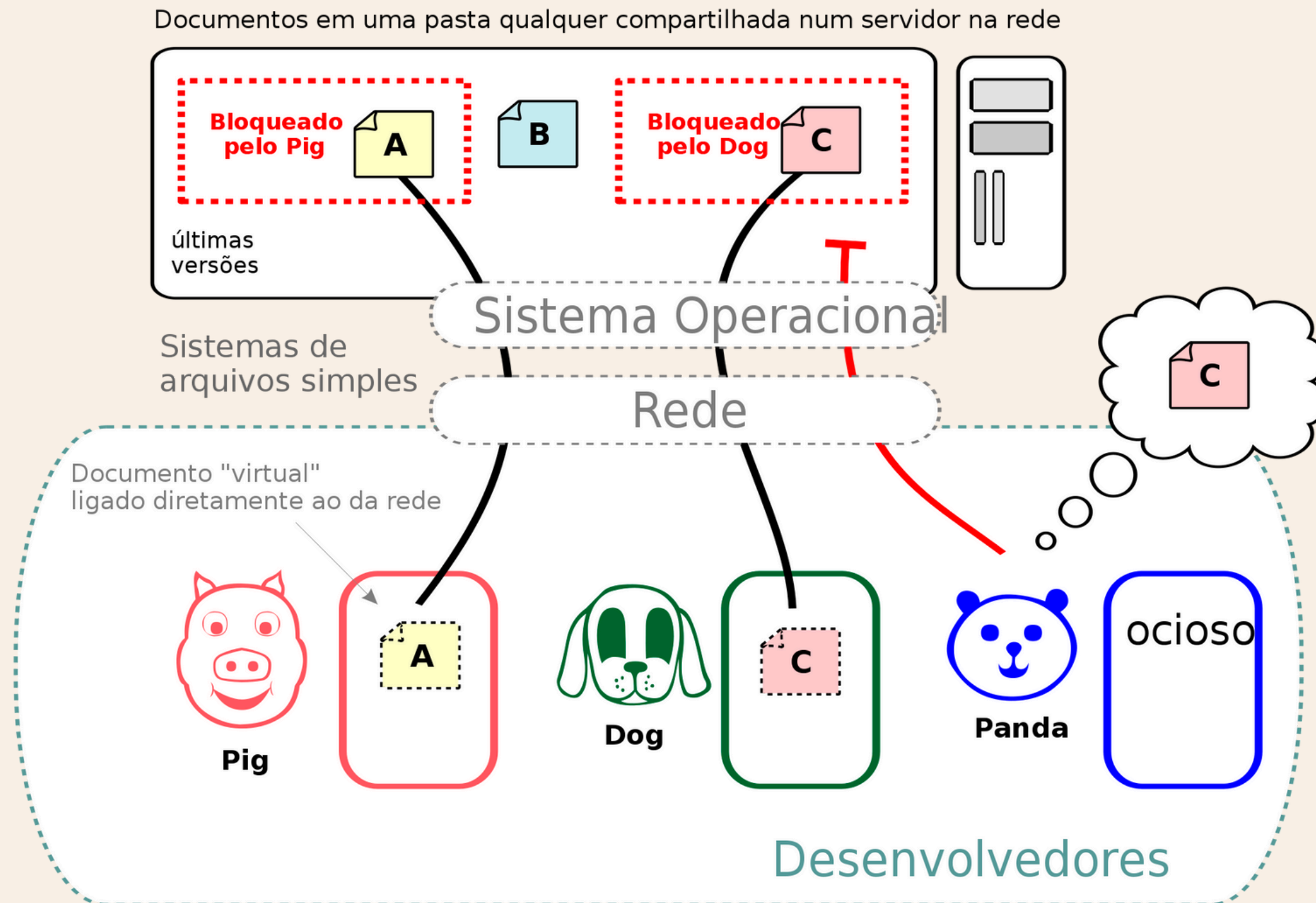
Controle de Versão - Branch and Merge



Controle de Versão - Branch and Merge



Controle de Versão - Exclusive Lock



Versionamento com GIT



01 Sistema de controle de versão distribuído.

02 Sistema de gerenciamento de código fonte.

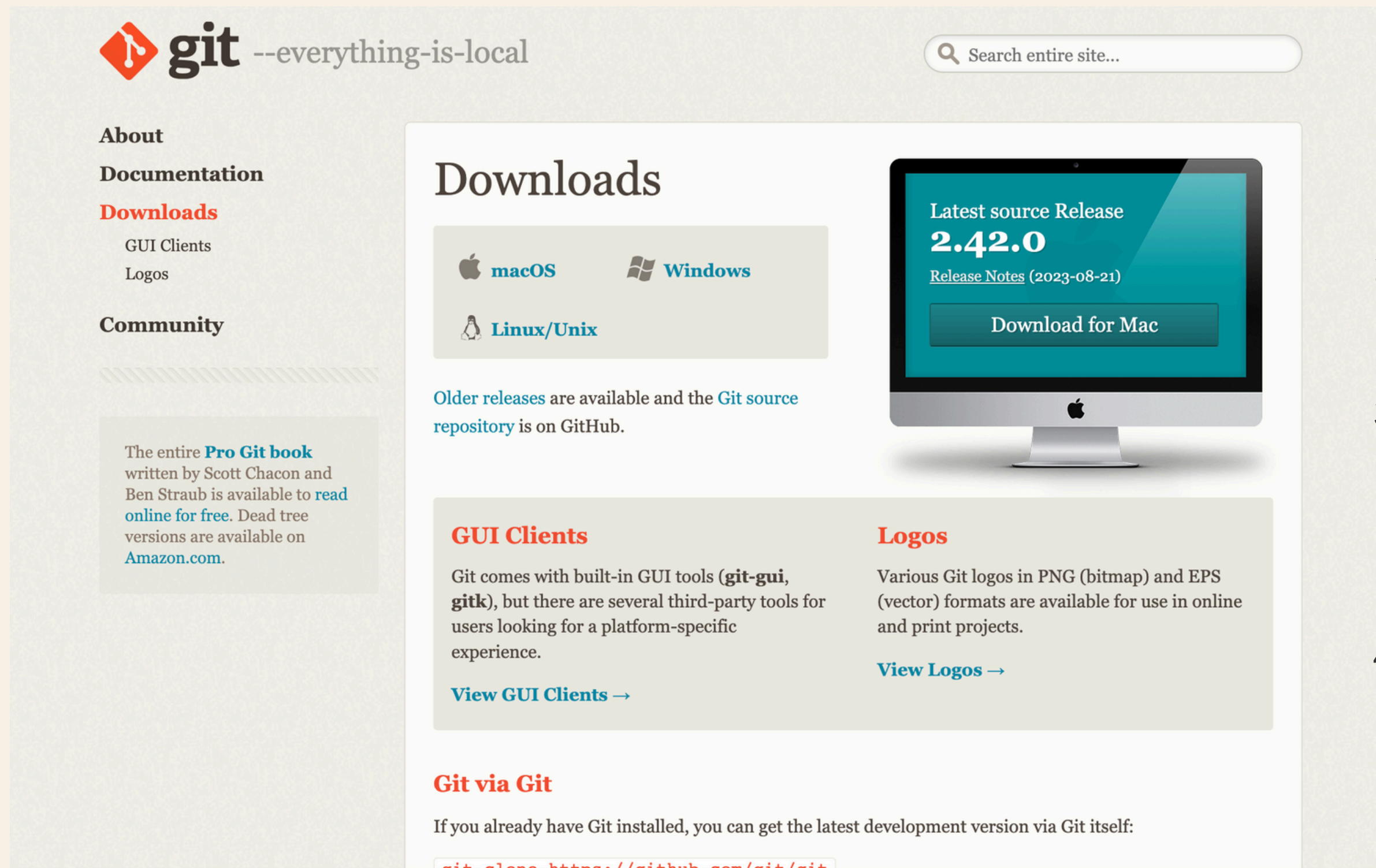
03 Desenvolvido por Linus Torvalds (criador do Linux).

04 Utilizado para criação de um histórico de alterações em código fonte.



<https://git-scm.com/download>

GIT - Instalação



The screenshot shows the Git website's 'Downloads' page. At the top left is the Git logo with the tagline '--everything-is-local'. A search bar is at the top right. The left sidebar contains links for 'About', 'Documentation', 'Downloads' (highlighted), 'GUI Clients', 'Logos', and 'Community'. The main content area features a 'Downloads' heading, a search bar, and a large image of a monitor displaying the latest source release '2.42.0' with a 'Download for Mac' button. Below this, there are sections for 'GUI Clients' and 'Logos'. The 'GUI Clients' section mentions built-in tools like 'git-gui' and 'gitk', and provides a link to 'View GUI Clients'. The 'Logos' section mentions available formats and provides a link to 'View Logos'. At the bottom, the 'Git via Git' section provides a command to clone the Git repository.

git --everything-is-local

Search entire site...

About

Documentation

Downloads

GUI Clients

Logos

Community

The entire **Pro Git book** written by Scott Chacon and Ben Straub is available to [read online for free](#). Dead tree versions are available on [Amazon.com](#).

Downloads

Latest source Release
2.42.0
[Release Notes \(2023-08-21\)](#)
Download for Mac

Older releases are available and the [Git source repository](#) is on GitHub.

GUI Clients

Git comes with built-in GUI tools (**git-gui**, **gitk**), but there are several third-party tools for users looking for a platform-specific experience.

[View GUI Clients →](#)

Logos

Various Git logos in PNG (bitmap) and EPS (vector) formats are available for use in online and print projects.

[View Logos →](#)

Git via Git

If you already have Git installed, you can get the latest development version via Git itself:

```
git clone https://github.com/git/git
```

1. Baixe o instalador:
 - Acesse o site oficial do Git: <https://git-scm.com/downloads> e escolha o seu sistema operacional.
2. Execute o instalador:
 - Após o download, clique duas vezes no arquivo para iniciar a instalação.
3. Siga as instruções:
 - Durante a instalação, você pode manter as configurações padrões.
 - Clique em "*Next*" até chegar ao final e, por fim, clique em "*Finish*".
4. Verifique a instalação
 - Abra o terminal.
 - Digite o comando: `git --version`
 - Se aparecer a versão do Git, a instalação foi bem-sucedida.

Conceitos Importantes



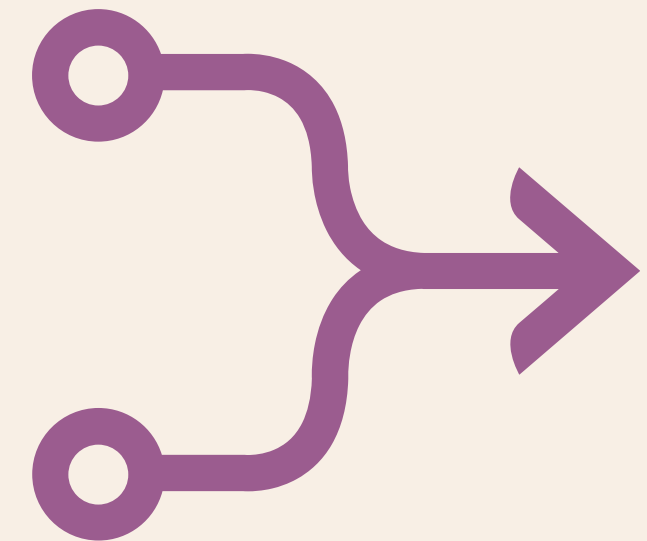
REPOSITÓRIO

- Local onde os arquivos e suas cópias serão armazenados.
- Local/Remoto



BRANCH

- Ramos, cópias do código original .
- Permite alterações de forma segura, sem afetar as funcionalidades.



MERGE

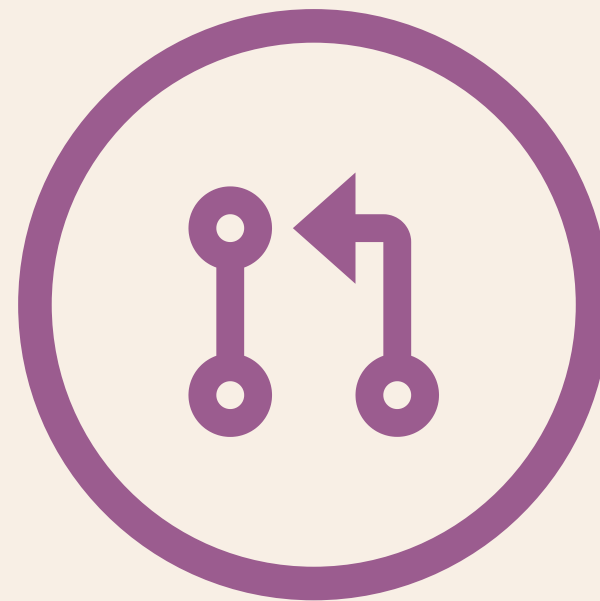
- Fundir a cópia com o ramo principal.

Conceitos Importantes



PUSH REQUEST

- Envio de modificações para o repositório central.



PULL REQUEST

- Obtenção das modificações para a sua máquina.



FORK

- Cópia de um repositório remoto para a máquina local.
- Pegar um código público e utilizá-lo.

Boas Práticas



Use commits descritivos

Explique o que foi alterado.



Mantenha commits pequenos e frequentes

Facilita o rastreamento de mudanças.



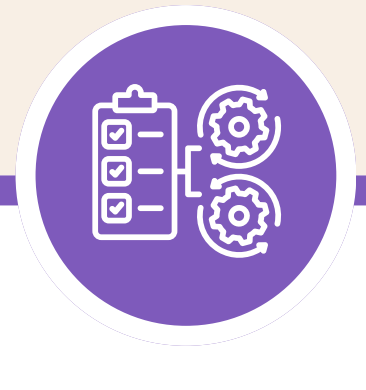
Utilize branches corretamente

Separe funcionalidades e correções em branches diferentes.



Revisão de código (Code Review)

Sempre revise código antes de mesclar branches.



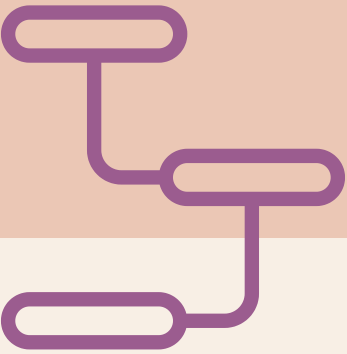
Automatize testes

Configure pipelines para testes automáticos.

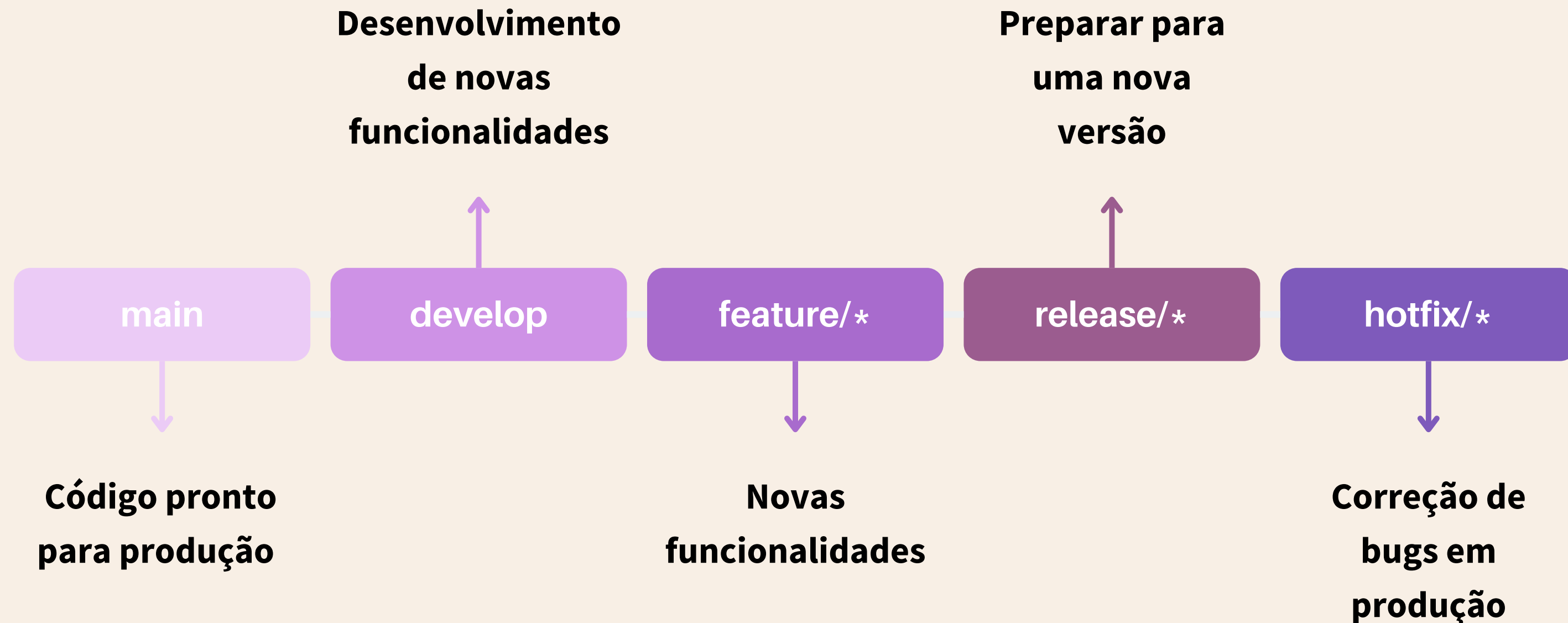
Fluxos de Trabalho Comuns - Básico



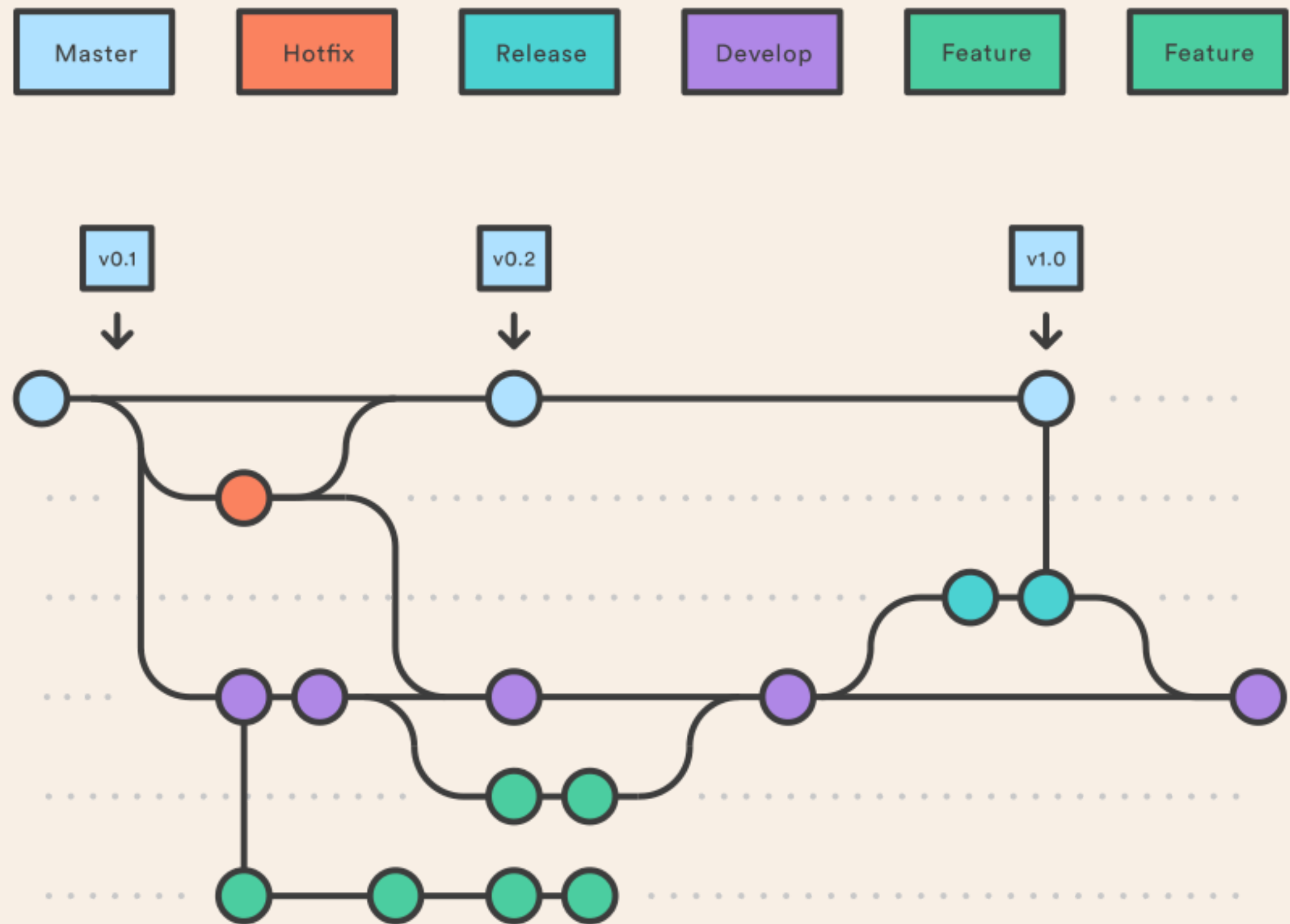
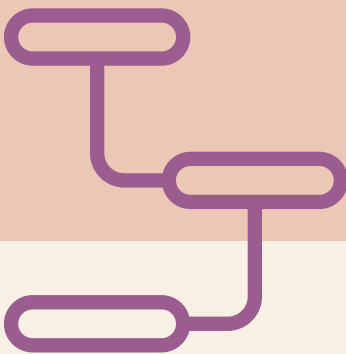
Fluxos de Trabalho Comuns - Git Flow



- ✓ Estrutura popular que define branches específicas para desenvolvimento, testes e produção.
- ✓ Principais branches



Fluxos de Trabalho Comuns - Git Flow



Comandos Básicos - Configuração



A primeira coisa que você deve fazer quando instalar o Git é definir o seu nome de usuário e endereço de e-mail.



Todos os commits no Git utilizam essas informações, e está imutavelmente anexado nos commits que você realiza.

```
git config --global user.name "Rafaela Moreira"  
git config --global user.email rafapcmor@gmail.com
```



Necessário configurar somente uma vez no computador.

Comandos Básicos - Configuração



Se você precisar de ajuda ao usar Git, existem três maneiras de obter a ajuda para qualquer um dos comandos Git.

```
git help {comando}  
git {comando} --help  
man git - {comando}
```

Comandos Básicos - Criando o projeto



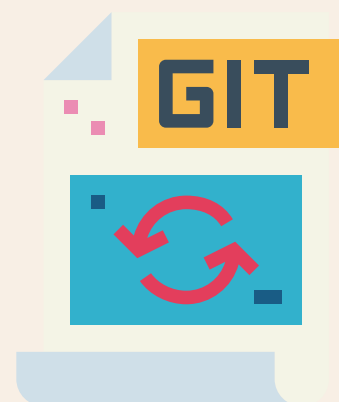
Você pode obter um projeto Git utilizando duas formas principais.



A primeira faz uso de um projeto ou diretório existente e o importa para o Git.



A segunda clona um repositório Git existente a partir de outro servidor.



Comandos Básicos - Criando o projeto



Caso você esteja iniciando o monitoramento de um projeto existente com Git, você precisa ir para o diretório do projeto e digitar

```
git init
```



Isso cria um novo subdiretório chamado .git que contem todos os arquivos necessários de seu repositório

Comandos Básicos - Criando o projeto



Caso você queira começar a controlar o versionamento dos arquivos existentes (diferente de um diretório vazio), você provavelmente deve começar a monitorar esses arquivos e fazer um commit inicial.

```
touch .gitignore  
git add .gitignore  
git commit -m "Versão inicial do projeto"
```

Comandos Básicos - Criando o projeto



Você clona um repositório remoto com:

```
git clone link_repositorio
```


Comandos Básicos - Status e mudanças



A principal ferramenta utilizada para determinar quais arquivos estão em quais estados é o comando:

```
git status
```



Se você quer saber exatamente o que você alterou, não apenas quais arquivos foram alterados , você pode utilizar o comando:

```
git diff
```


Comandos Básicos - Gravando alterações



Quando um repositório é inicialmente clonado, todos os seus arquivos estarão monitorados e inalterados porque você simplesmente os obteve e ainda não os editou. Conforme você edita esses arquivos, o Git passa a vê-los como modificados, porque você os alterou desde seu último commit.



Para passar a monitorar um novo arquivo, use o comando `git add`

```
git add README
```

Comandos Básicos - Commit



Armazena o conteúdo atual do índice em um novo commit, juntamente com uma mensagem de registro do usuário que descreve as mudanças.

Se usa o commit depois de já ter feito o git add, para fazer o commit:

```
git commit -m "Mensagem"
```

Comandos Básicos - Desfazendo mudanças



Em qualquer fase, você pode querer desfazer alguma coisa. Essa é uma das poucas áreas no Git onde você pode perder algum trabalho se fizer errado.

Para voltar ao último commit:

```
git reset --hard HEAD~1
```

Para voltar ao último commit e mantém os últimos arquivos no Stage:

```
git reset --soft HEAD~1
```

Comandos Básicos - Desfazendo mudanças



Volta para o commit com a hash XXXXXXXXXXXX:

```
git reset --hard XXXXXXXXXXXX
```

Comandos Básicos - Recuperando commit



Para visualizar os hashes

```
git reflog
```

E para aplicar:

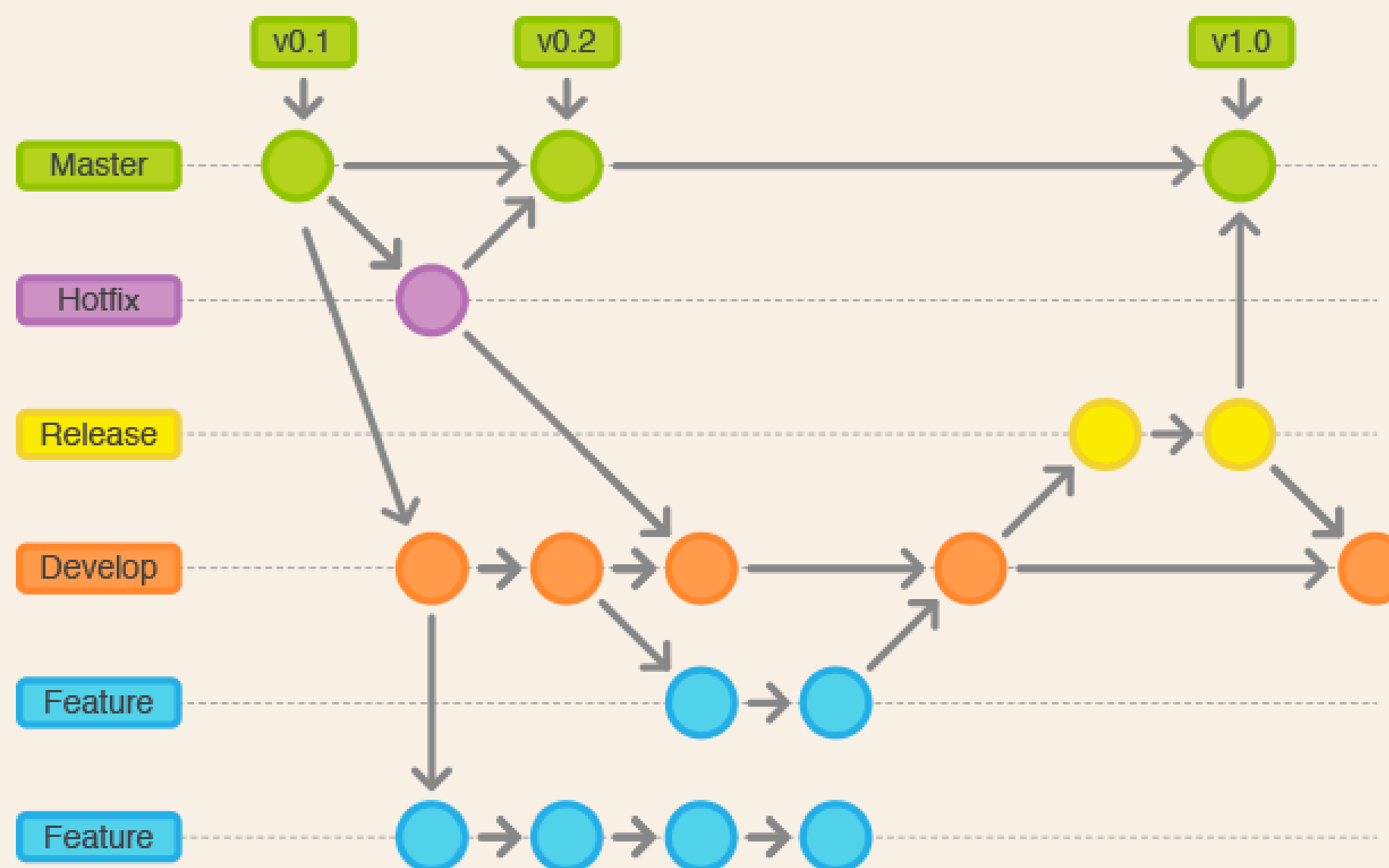
```
git merge {hash}
```


Comandos Básicos - Branch



Master: versão de produção.

Developer: versão em desenvolvimento + novas features



Comandos Básicos - Branch



- ✓ Um branch no Git é simplesmente um leve ponteiro móvel para um dos commits.
- ✓ O nome do branch padrão no Git é master.
- ✓ Para criar um novo branch, use o comando git branch:

```
git branch teste
```


Comandos Básicos - Branch



Para mudar para um branch existente, você executa o comando git checkout. Vamos mudar para o novo branch teste:

```
git checkout teste
```

Comandos Básicos - Merge



Suponha que você decidiu que o trabalho na tarefa #53 está completo e pronto para ser feito o merge no branch master. Tudo que você tem a fazer é executar o checkout do branch para onde deseja fazer o merge e então rodar o comando git merge:

```
git checkout master  
git merge iss53
```

Comandos Básicos - Histórico



Depois que você tiver criado vários commits, ou se clonou um repositório com um histórico de commits existente, você provavelmente vai querer ver o que aconteceu. A ferramenta mais básica e poderosa para fazer isso é o comando:

```
git log
```

Comandos Básicos - Compartilhar e atualizar



Para pegar dados dos seus projetos remotos. Esse comando vai até o projeto remoto e pega todos os dados que você ainda não tem. Depois de fazer isso, você deve ter referências para todos os branches desse remoto, onde você pode fazer o merge ou inspecionar a qualquer momento.

```
git fetch origin
```

Comandos Básicos - Compartilhar e atualizar



Incorpora as alterações de um repositório remoto no branch atual.

```
git pull
```



O git push é o comando em que você transfere commits a partir do seu repositório local para um repositório remoto.

```
git push
```


GitHub



- ✓ GitHub é uma **plataforma de hospedagem de código** para controle de versão e colaboração. Permite que você e outras pessoas trabalhem em conjunto em projetos de qualquer lugar.
- ✓ Ele fornece ferramentas poderosas para **colaboração**, **rastreamento** de problemas e **integração contínua**, tornando a colaboração em projetos de software mais rápida e eficiente.
- ✓ + de 150 milhões de desenvolvedores moldando juntos o futuro do software.



GitHub - Gerenciamento de Projetos



- ✓ O Projects é uma ferramenta adaptável e flexível para planejar e acompanhar o trabalho no GitHub.

Gerenciamento de Projetos

Mantenha solicitações de recursos, bugs e muito mais organizados com GitHub Issues – projetados para equipes de software.

Coordene iniciativas grandes e pequenas com tabelas de projetos, quadros e listas de tarefas.

GitHub



Welcome to GitHub!
Let's begin the adventure

Enter your email*



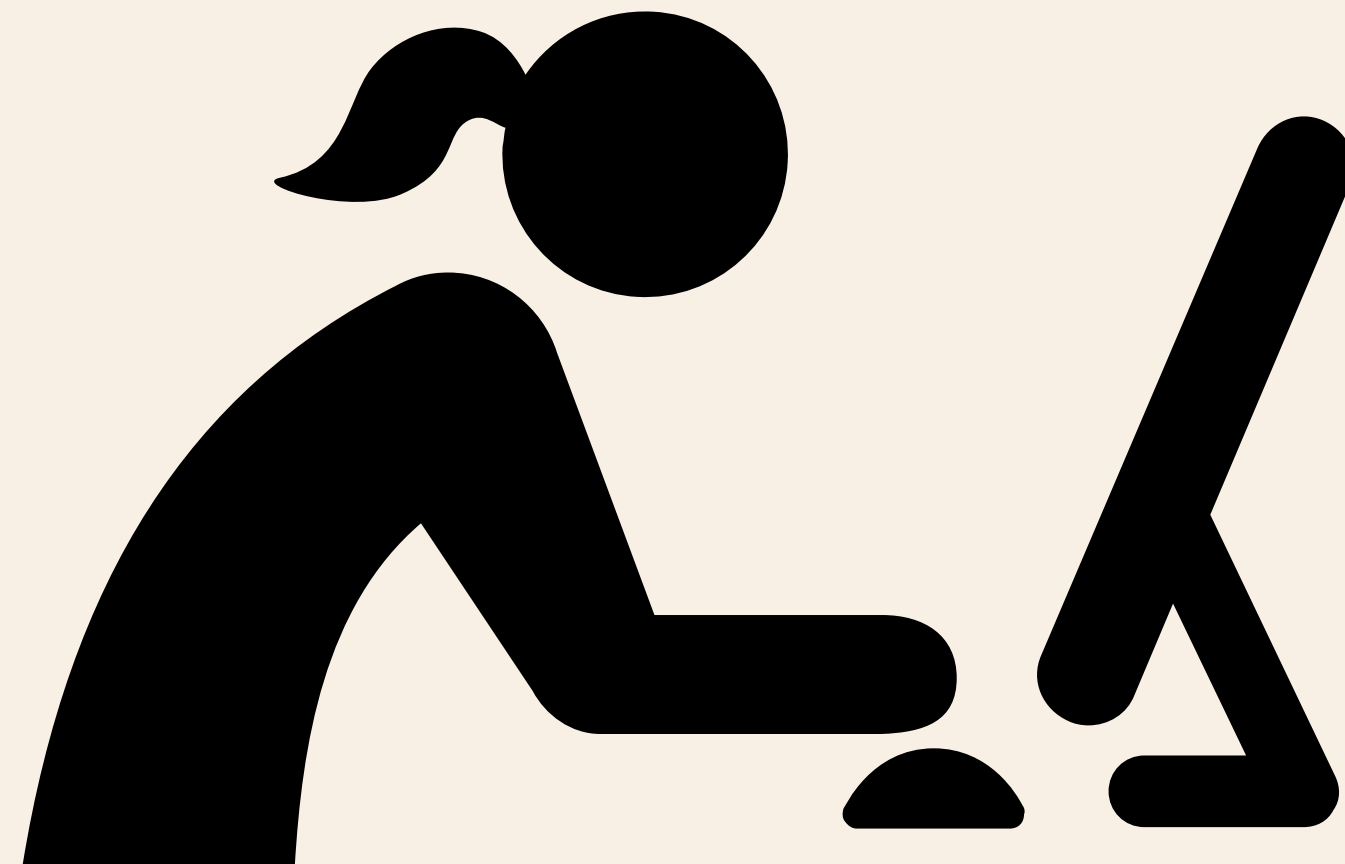
Continue

Documentação - Git e GitHub

©

©

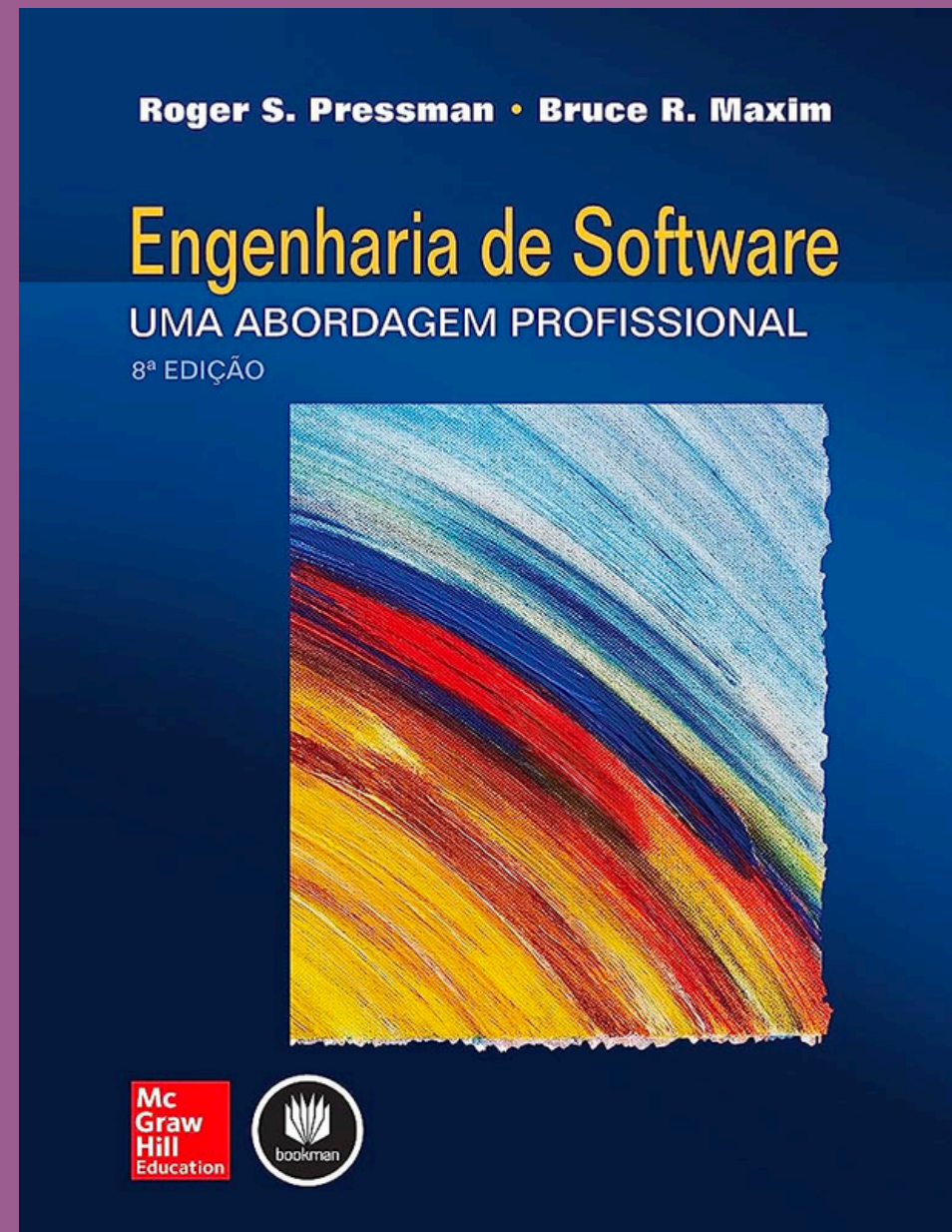
GitHub na Prática



DÚVIDAS?



Referências Bibliográficas



PROFESSORA: RAFAELA PRISCILA CRUZ MOREIRA

CONTATOS



rafaela.cruz@ulife.com.br



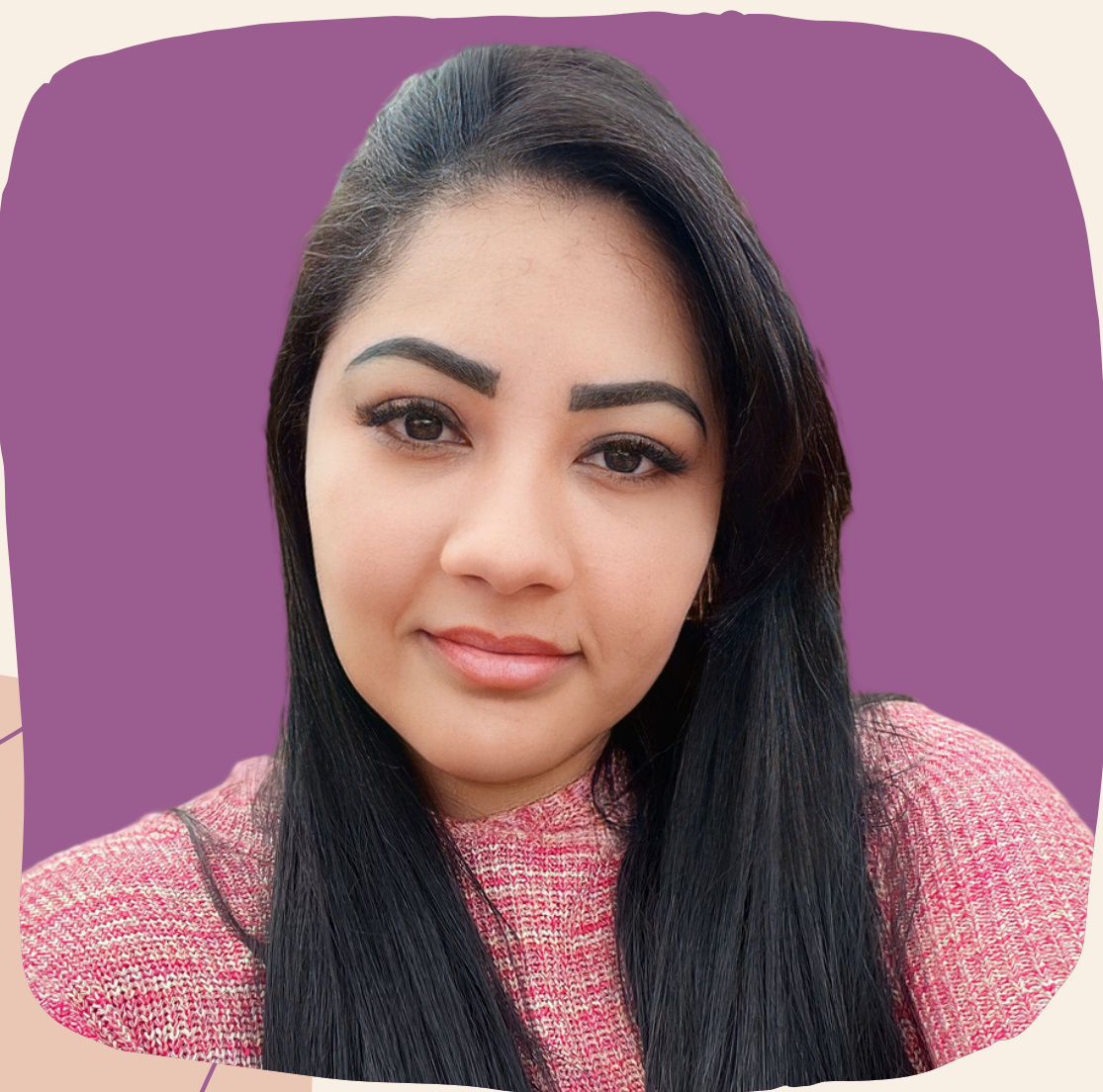
[rafaelapcmoreira](#)



[rafaelapcm](#)



[rafapcmor](#)





Thank You